

Gerenciamento da CPU

Nesta seção, exploraremos como o sistema operacional (SO) oferece a ilusão de que vários programas estão rodando simultaneamente, mesmo com um número limitado de CPUs. Vamos abordar como os usuários podem iniciar e parar programas usando as APIs do SO, e como o SO escolhe quais programas serão executados e quando serão alternados.

Escalonador

Quando o número de processos prontos para executar é maior do que o número de núcleos da CPU disponíveis, os PCBs (Process Control Blocks) dos processos prontos são mantidos em uma [fila de prontos](#).

O [escalonador do SO](#) é responsável por escolher o próximo processo a ser executado.

Suposições de Carga de Trabalho

Para simplificar a análise das políticas de escalonamento, vamos definir algumas suposições:

1. Todos os processos executam pelo mesmo período de tempo, e esse tempo é conhecido.
2. Todos os processos chegam ao sistema ao mesmo tempo.
3. Uma vez iniciado, cada processo é executado até a conclusão.
4. Os processos usam apenas a CPU (ou seja, nenhuma E/S é realizada).

É importante notar que a quantidade de tempo que um processo usa a CPU também é conhecida como [CPU Burst](#) do processo.

Métricas

Para avaliar a eficácia dos algoritmos de escalonamento, definimos algumas métricas.

[Tempo de retorno \(Turnaround time\)](#): O tempo que o processo leva para completar após chegar ao sistema.

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

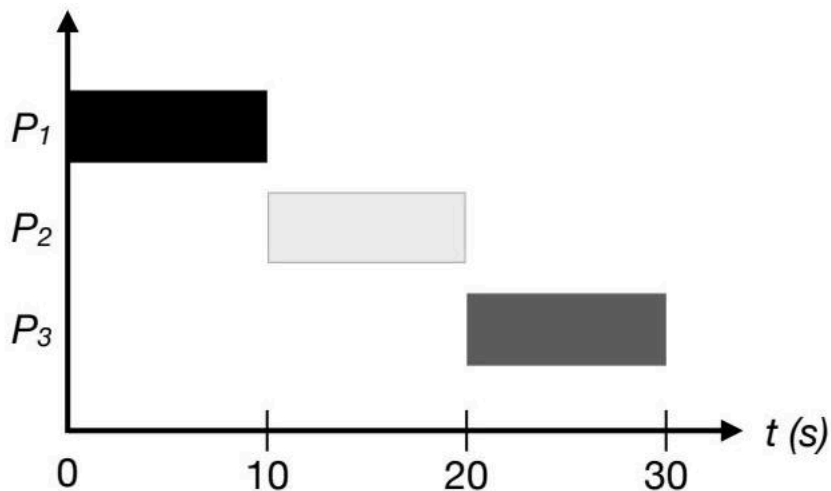
Além do tempo de retorno, outras métricas incluem:

- [Utilização da CPU](#): O SO busca manter a CPU ocupada executando processos.
- [Equidade \(Fairness\)](#): Todos os processos têm a chance de serem executados.

First Come, First Served (FCFS) / First In, First Out (FIFO)

- [Critério de seleção](#): Escalonar o processo que chegou primeiro (mais cedo).

Por exemplo, considere os processos P1, P2 e P3, cada um levando 10 segundos para executar. Se a ordem de chegada for P1 → P2 → P3 (assumindo que todos chegam no instante $t = 0$, com um pequeno atraso entre eles), podemos visualizar a execução usando um gráfico de Gantt:



O gráfico de Gantt acima ilustra o tempo de execução de cada processo, mostrando P1 executando primeiro, seguido por P2 e, finalmente, P3. O tempo total de cada processo é de 10 segundos, resultando em um tempo de resposta escalonado.

Nesse caso, o tempo médio de retorno é:

$$(10 + 20 + 30) / 3 = 20 \text{ segundos}$$

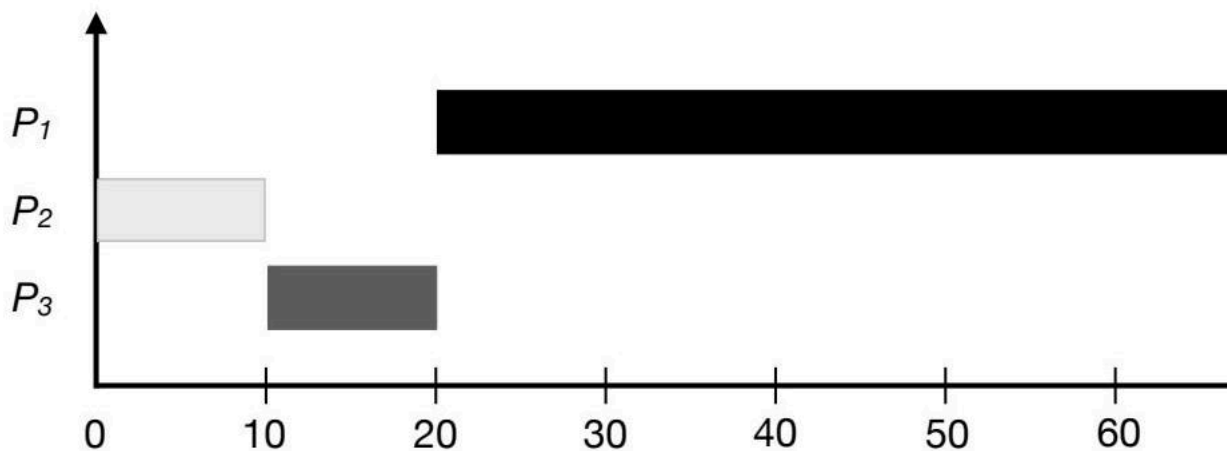
Processos com Tempos de Execução Distintos

Vamos relaxar a primeira suposição de que todos os processos são executados pelo mesmo período de tempo.

Bom cenário

Imagine que P1 seja executado por 100 segundos, enquanto P2 e P3 são executados por 10 segundos. Se a ordem de chegada for P2 → P3 → P1 (todos chegando em t = 0, com um pequeno atraso entre eles), o tempo médio de retorno seria:

$$(10 + 20 + 120) / 3 = 50 \text{ segundos}$$

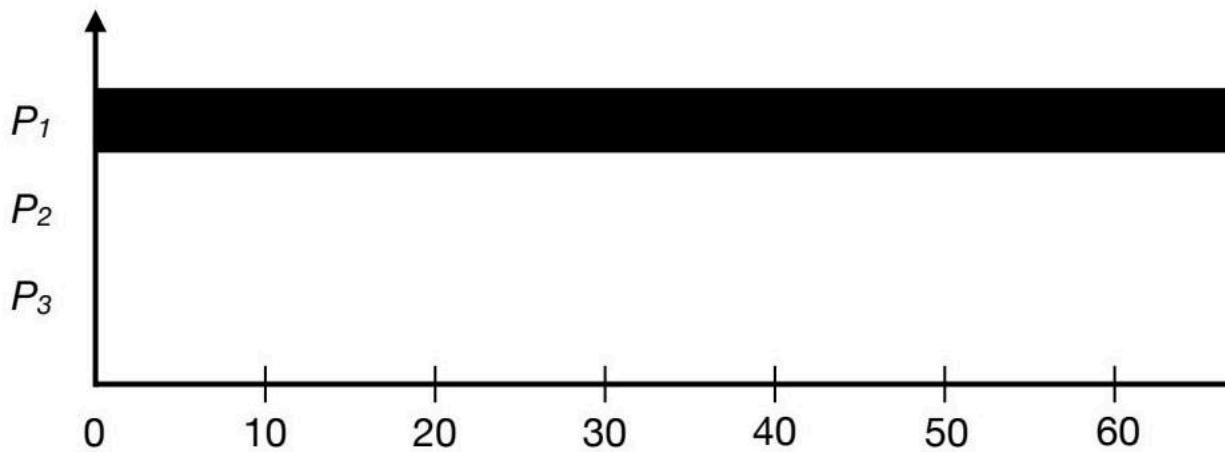


O gráfico de Gantt mostra P2 e P3 completando rapidamente, seguidos por P1, que leva muito mais tempo. Isso afeta o tempo médio de resposta.

Cenário ruim

Agora, se a ordem de chegada for $P_1 \rightarrow P_2 \rightarrow P_3$, o tempo médio de retorno seria:

$$(100 + 110 + 120) / 3 = 110 \text{ segundos}$$



O gráfico mostra P1 sendo executado por um longo período, atrasando a execução de P2 e P3. Isso resulta em um tempo de retorno médio muito maior.

Efeito Comboio (Convoy Effect)

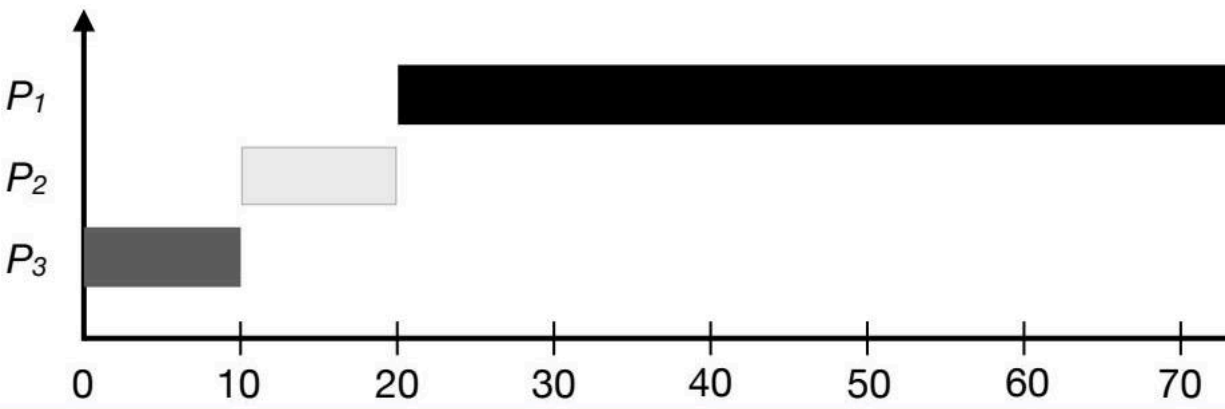
O **efeito comboio** ocorre quando processos curtos ficam enfileirados atrás de um processo pesado (de longa duração).

Shortest Job First (SJF)

- **Critério de seleção:** Execute o processo com o menor tempo de execução, depois o próximo menor e assim por diante.

Se P1 executar por 100 segundos, enquanto P2 e P3 forem executados por 10 segundos e a ordem de chegada for $P_1 \rightarrow P_3 \rightarrow P_2$, o tempo médio de retorno seria:

$$(10 + 20 + 120) / 3 = 50 \text{ segundos}$$



Este gráfico mostra a execução de P2 e P3 primeiro devido ao seu menor tempo de execução, seguido por P1. Isso ajuda a minimizar o tempo médio de resposta.

Processos Chegando em Tempos Diferentes

Vamos relaxar a segunda suposição de que todos os processos chegam ao sistema ao mesmo tempo.

Preempção

Por definição, SJF é um **escalonador não preemptivo**, o que significa que os processos são executados até que cedam a CPU.

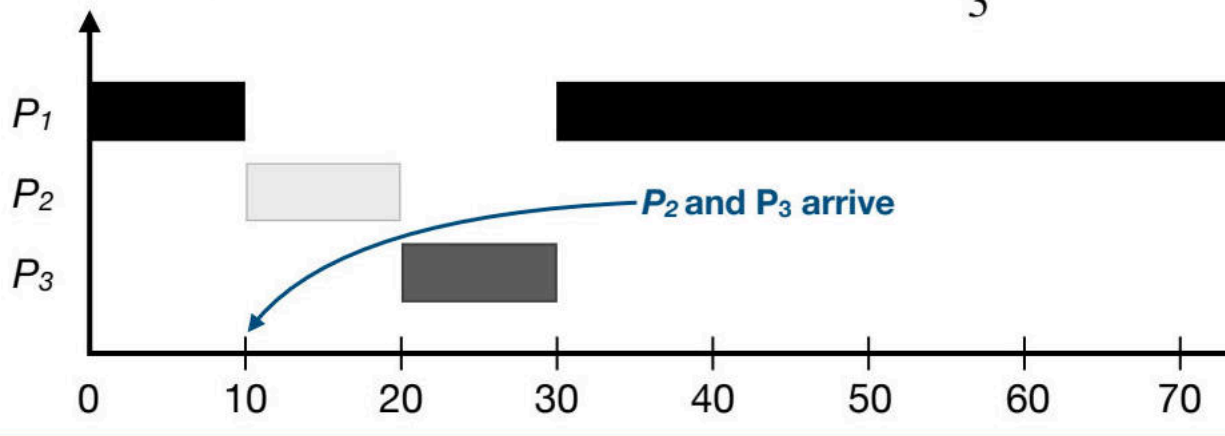
Para aliviar o efeito comboio anterior, o escalonador deve **preemptar** P1 e executar outro processo.

Shortest Time-to-Completion First (STCF) / Shortest Remaining Time (SRT)

- **Critério de seleção:** Sempre que um novo processo entra no sistema, escalonar o processo com o menor tempo de execução restante.

Se P1 executar por 100 segundos, enquanto P2 e P3 forem executados por 10 segundos e a ordem de chegada for: P1 chega no instante $t = 0$, enquanto P2 e P3 chegam no instante $t = 10$, STCF deve preemptar P1 quando P2 e P3 chegarem.

► **Average turnaround time:** $\frac{(120 - 0) + (20 - 10) + (30 - 10)}{3}$



O gráfico ilustra como STCF gerencia os processos, priorizando aqueles com o tempo de conclusão mais curto, o que resulta em um tempo médio de resposta mais baixo.

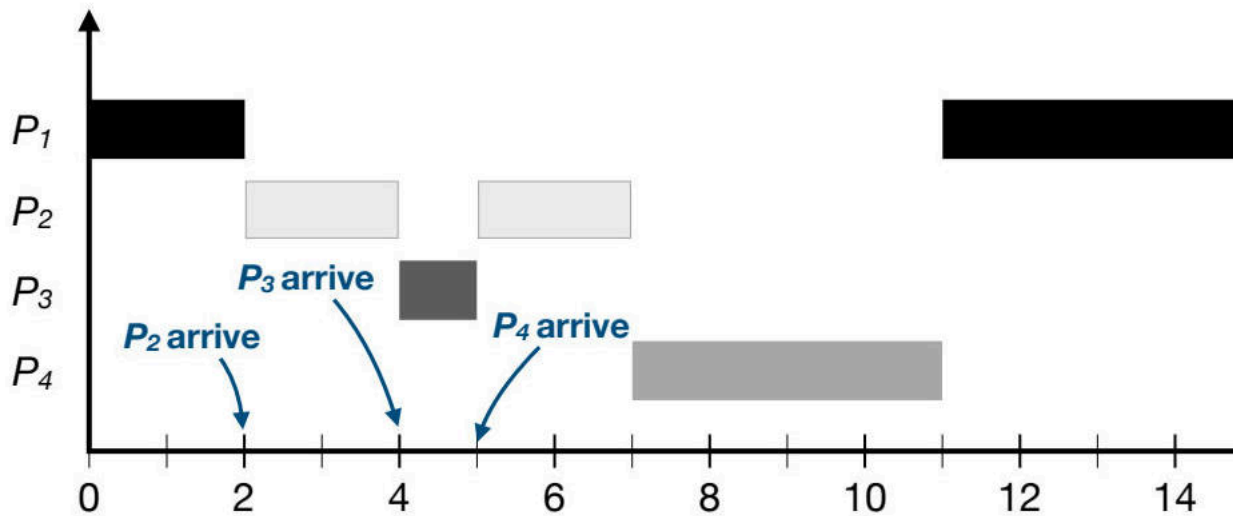
Outro Exemplo de STCF

Supondo os tempos de chegada e execução na tabela abaixo:

Processo	Chegada	Tempo de Execução (CPU Burst)
1	0	7
2	2	4
3	4	1
4	5	4

O tempo médio de retorno é:

$$(16 - 0) + (7 - 2) + (5 - 4) + (11 - 5) / 4 = 7.75 \text{ segundos}$$



Este gráfico mostra como os processos são escalonados com base no tempo restante, resultando em um tempo médio de resposta eficiente.

Tempo de Resposta em Sistemas Interativos

Com computadores de tempo compartilhado, os usuários começaram a pedir desempenho interativo e, portanto, surge uma nova métrica:

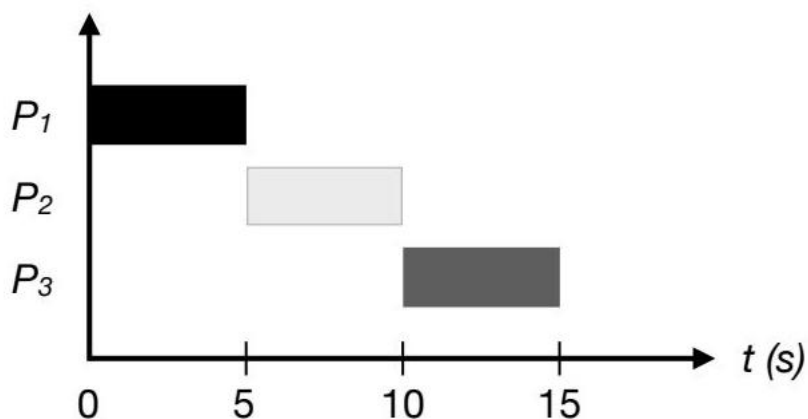
Tempo de resposta: Tempo desde quando o processo chega ao sistema até a primeira vez que é escalonado.

$$T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$$

Exemplo com SJF em Sistemas Interativos

Considere os processos P1, P2 e P3, cada um levando 5 segundos para executar. Se a ordem de chegada for P1 → P2 → P3 (todos chegando em $t = 0$, com um pequeno atraso), o tempo médio de resposta seria:

$$(0 + 5 + 10) / 3 = 5 \text{ segundos}$$



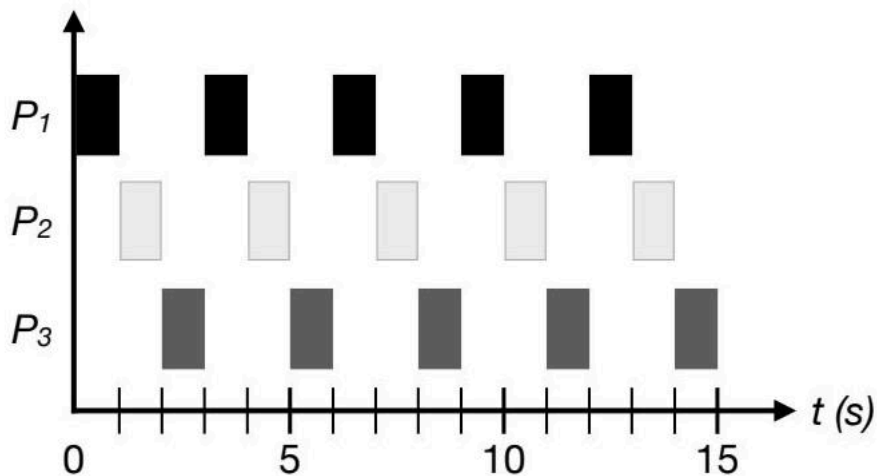
O gráfico mostra como SJF escala os processos, afetando o tempo de resposta em um ambiente interativo.

Round Robin (RR) / Time-Slicing

- **Critério de seleção:** Execute um processo por um determinado intervalo de tempo (quantum de escalonamento) e, em seguida, alterne para o próximo processo na fila (os processos enfileirados são atendidos em um estilo FCFS).

Se P1, P2 e P3 levarem 5 segundos para executar, a ordem de chegada for P1 → P2 → P3 (todos chegando no instante 0, com um pequeno atraso) e o quantum de escalonamento for 1 segundo, o tempo médio de resposta seria:

$$(0 + 1 + 2) / 3 = 1 \text{ segundo}$$



O gráfico ilustra como RR divide o tempo da CPU entre os processos, o que resulta em um tempo de resposta melhor.

Considerações sobre RR

O **quantum de escalonamento** é crucial para a eficiência do RR:

- **Valores curtos:** Melhor tempo de resposta.
- **Valores grandes:** Semelhante a uma política FCFS.

Um quantum de escalonamento muito pequeno pode ser problemático:

- O custo (tempo gasto) da **troca de contexto** pode dominar o desempenho geral.
- A duração do quantum de escalonamento deve amortizar o custo da troca de contexto.

Por exemplo:

- quantum de escalonamento = 10 ms e troca de contexto = 1 ms - 10% de tempo desperdiçado.
- quantum de escalonamento = 100 ms e troca de contexto = 1 ms - 1% de tempo desperdiçado.

Tempo de Retorno em RR

Se P1, P2 e P3 levarem 5 segundos para executar:

Example: Processes P_1, P_2 and P_3 , each take 15 seconds to execute.

- ▶ **Order of arrival:** $P_1 \rightarrow P_2 \rightarrow P_3$ (all arrived at the same time)
- ▶ **Scheduling quantum:** 1 second
- ▶ **Average turnaround time:** $\frac{13 + 14 + 15}{3} = 14 \text{ seconds}$

O tempo médio de retorno é:

$$(13 + 14 + 15) / 3 = 14 \text{ segundos}$$

Outro Exemplo de RR

Supondo os seguintes tempos de chegada e execução:

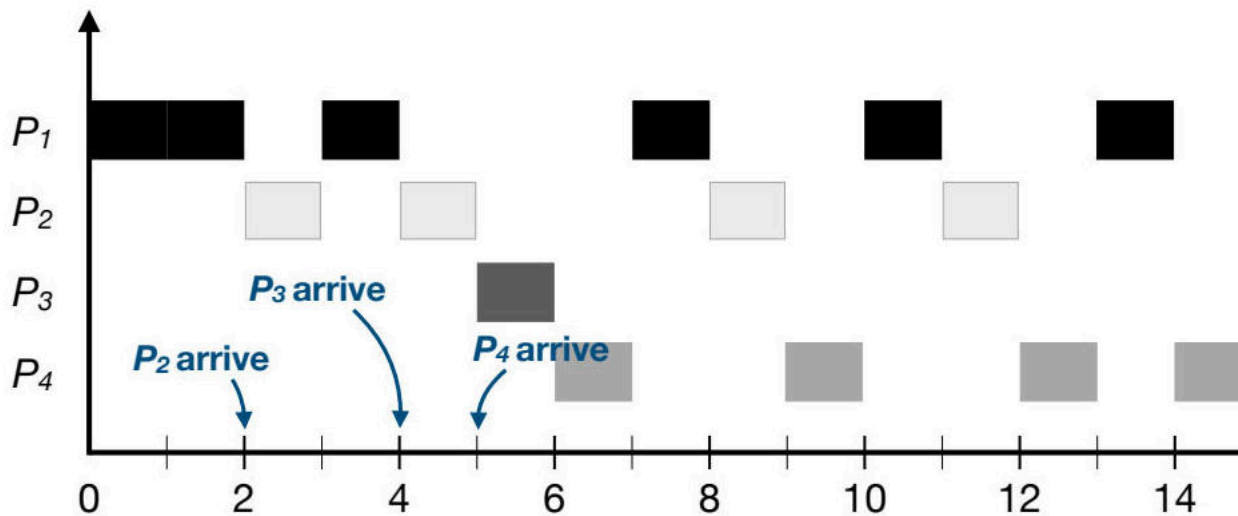
Processo	Chegada	Tempo de Execução (CPU Burst)
1	0	7
2	2	4
3	4	1
4	5	4

Tempo médio de resposta:

$$(0) + (2 - 2) + (5 - 4) + (7 - 6) / 4 = 0.5 \text{ segundos}$$

Tempo médio de retorno:

$$(16 - 0) + (12 - 2) + (6 - 4) + (15 - 5) / 4 = 9.5 \text{ segundos}$$



Resumo de FCFS, SJF, STCF e RR

Algoritmo	Vantagens	Desvantagens
FCFS	Simple e fácil de implementar.	O tempo de retorno é muito sensível à ordem de chegada dos processos.
SJF/STCF	Melhoram o tempo de retorno; STCF é melhor quando os processos chegam em horários diferentes.	Requerem conhecimento prévio do tempo de execução e são ruins ao considerar o tempo de resposta.
RR	Escalonador justo, bom para melhorar o tempo de resposta (importante para sistemas interativos).	Um dos piores algoritmos para tempo de retorno (algoritmos justos são ruins para essa métrica).

Existe um tradeoff entre otimizar o tempo de retorno e o tempo de resposta.

Decisões do Escalonador ao Considerar E/S

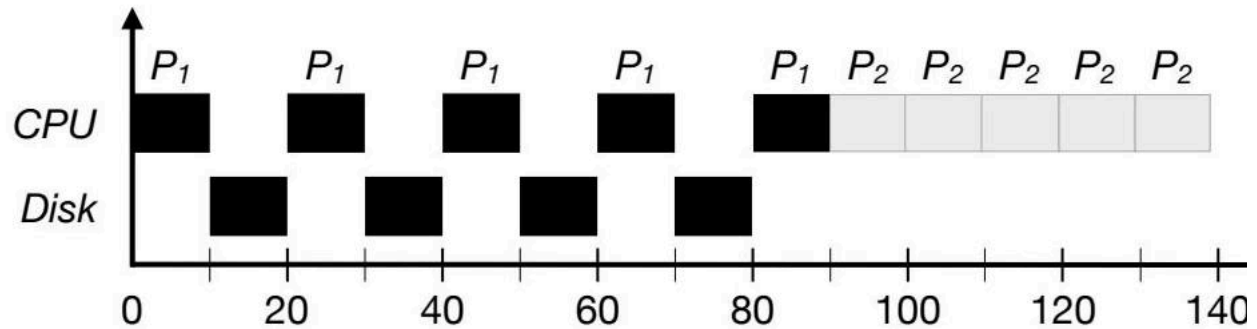
Vamos relaxar a quarta suposição de que os processos usam apenas a CPU (ou seja, nenhuma E/S é feita).

Decisões do Escalonador

- Se um processo bloqueia para E/S, outro processo deve ser escalonado?
- Quando a E/S é concluída, o processo correspondente deve ser retomado imediatamente?

Abordagem Ingênua - Sem Sobreposição ao Considerar E/S

Suponha que os processos P1 e P2 executem por 50 ms, P1 execute operações de E/S a cada 10 ms e P2 use apenas a CPU. O escalonador executa P1 e depois P2.

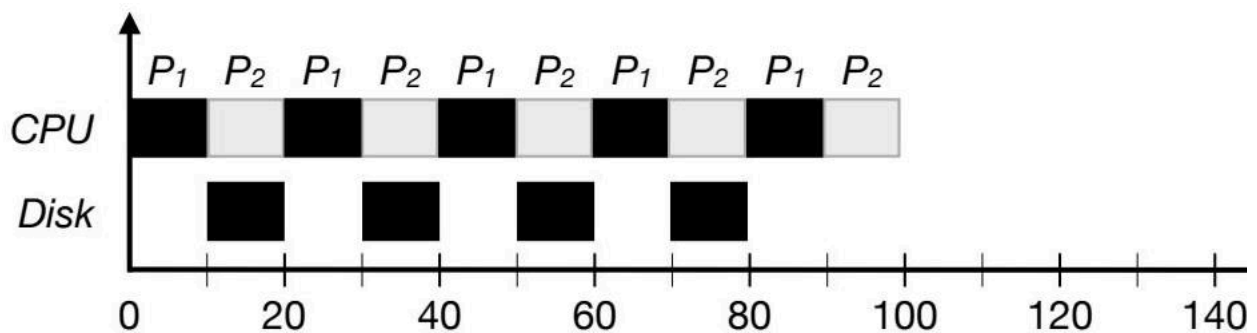


Sobreposição de Recursos ao Considerar E/S

Vamos supor um algoritmo baseado em STCF, mas:

- Cada fatia de tempo de 10 ms de P1 é tratada como um processo independente.
- Processos interativos (bloqueando para E/S) são executados com frequência.
- Processos intensivos em CPU são executados enquanto os interativos bloqueiam para E/S (sobreposição).

Isso resulta em melhores tempos de resposta e retorno.



Multi-Level Feedback Queue (MLFQ)

Vamos relaxar nossa última suposição principal: o tempo de execução dos processos é conhecido.

Objetivos do SO

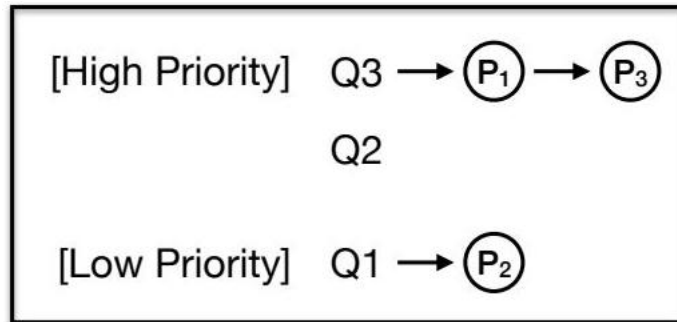
- Otimizar o tempo de retorno, como em SJF ou STCF (mas sem conhecer o tempo de execução dos processos).
- Fazer com que o sistema pareça responsivo para usuários interativos, como em RR (bom tempo de resposta).

Solução

O escalonador deve combinar várias políticas de escalonamento e aprender com o passado para tomar melhores decisões no futuro.

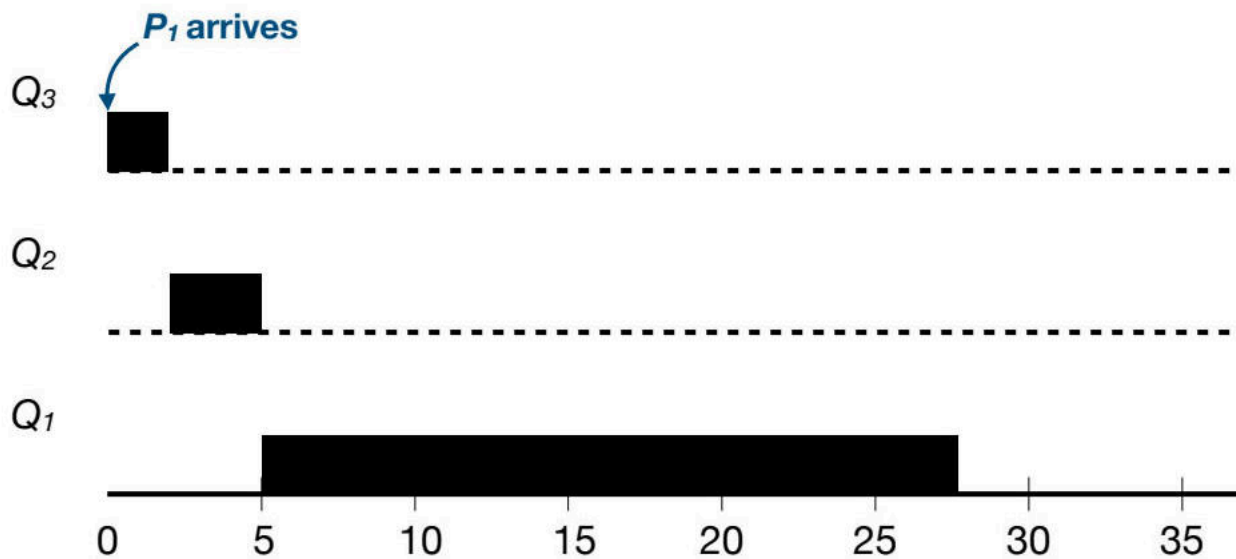
Regras Básicas do MLFQ

- Múltiplas filas, ordenadas de acordo com sua prioridade.
- As prioridades são usadas para escolher qual processo é executado em seguida.
 - Regra 1: Se $Prioridade(P1) > Prioridade(P2)$, P1 é executado (P2 não).
 - Regra 2: Se $Prioridade(P1) = Prioridade(P3)$, P1 e P3 são executados em RR.



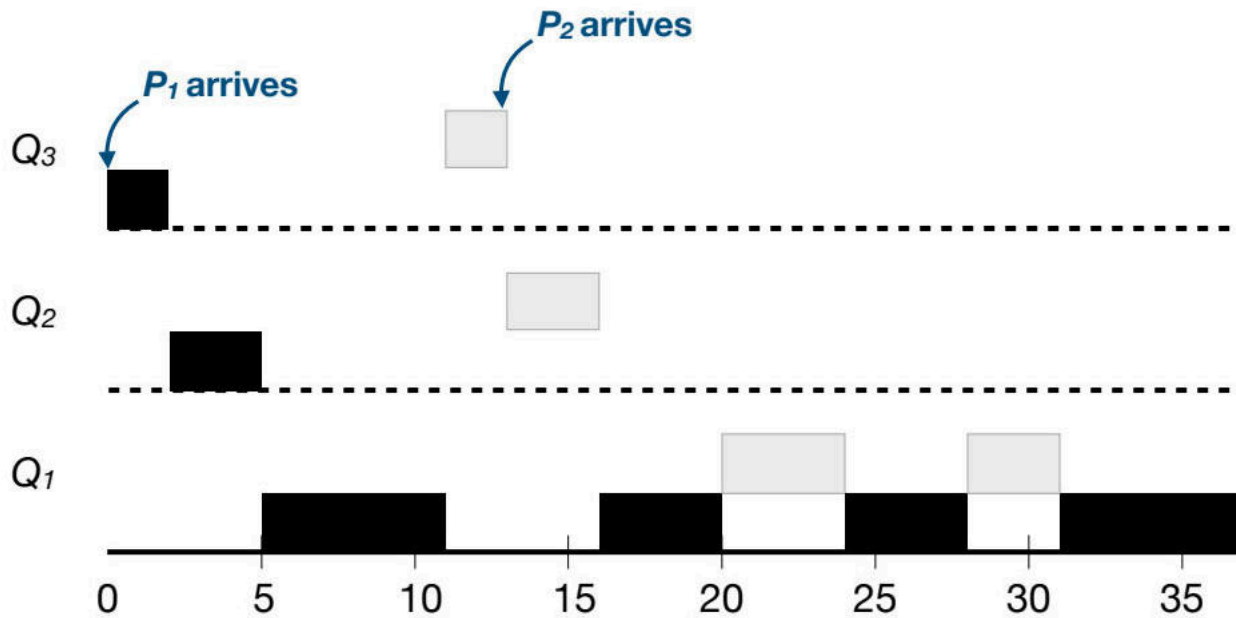
Movimento da Fila no MLFQ

- Regra 3: Quando um processo entra no sistema, ele tem a maior prioridade (fila superior).
- Regra 4: Depois que um processo usa todo o seu tempo alocado em um determinado nível, sua prioridade é reduzida (ou seja, move-se para baixo uma fila).



Preempção de Processos de Baixa Prioridade no MLFQ

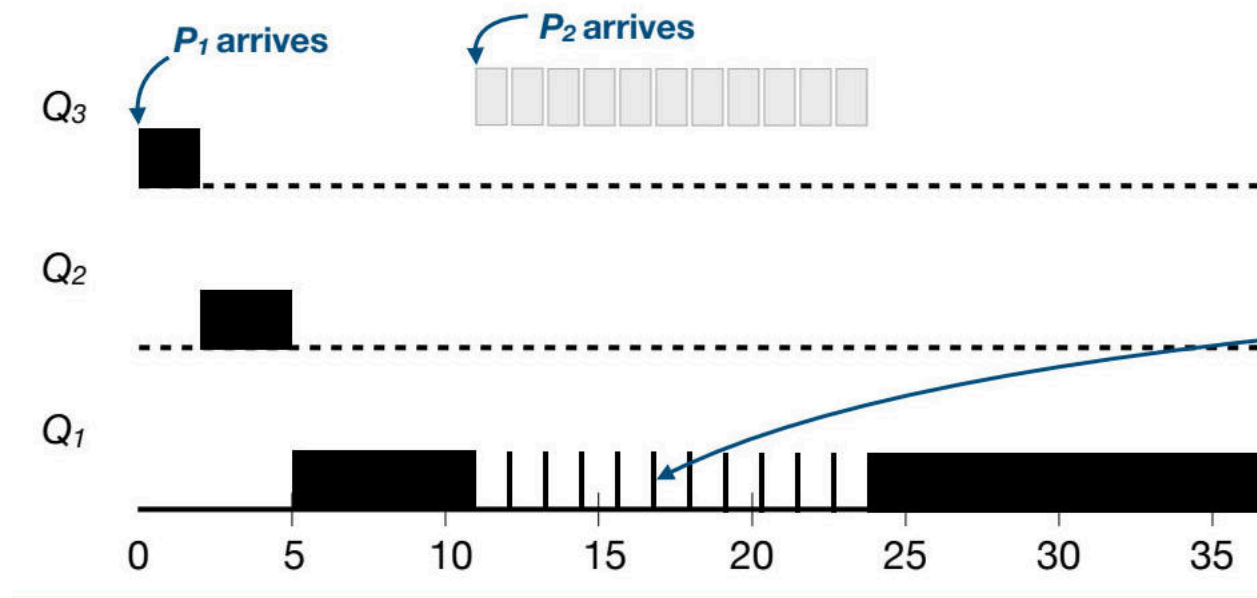
Quando novos processos, com maior prioridade, chegam, os processos de baixa prioridade em execução são preemptados.



Gaming no MLFQ

Processos interativos (por exemplo, P₂) podem ceder a CPU (bloquear para E/S) antes que seu tempo alocado expire. A prioridade deles deve ser mantida?

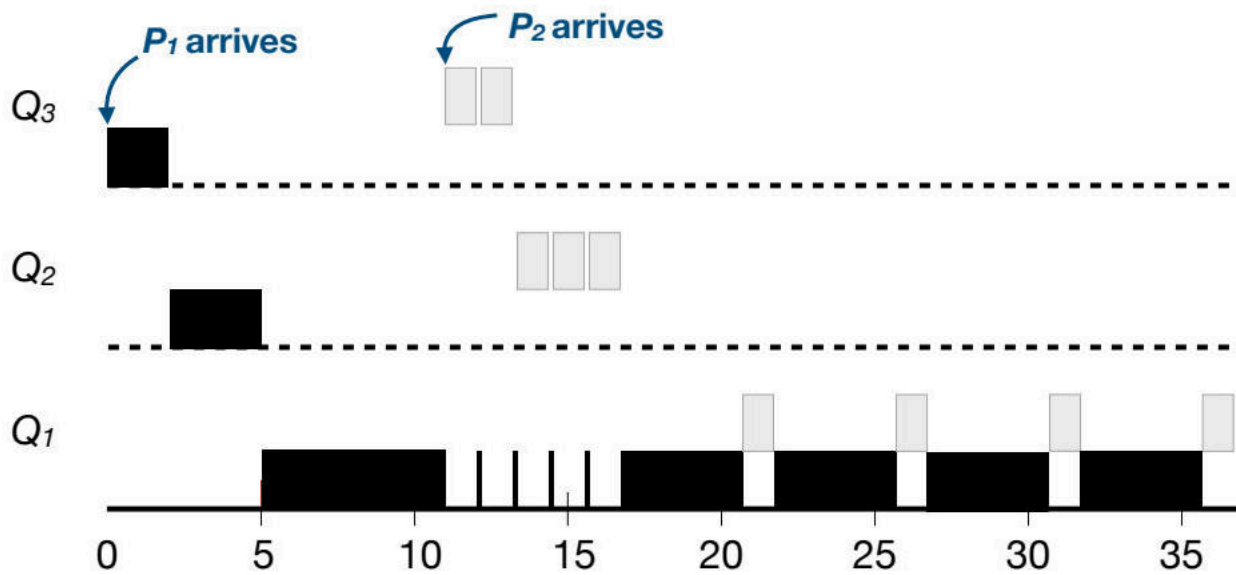
Usuários ruins podem enganar (jogar) o SO e prejudicar outros processos escrevendo um programa que usa repetidamente 99% do tempo alocado e, em seguida, cede a CPU.



Melhor Contabilidade no MLFQ

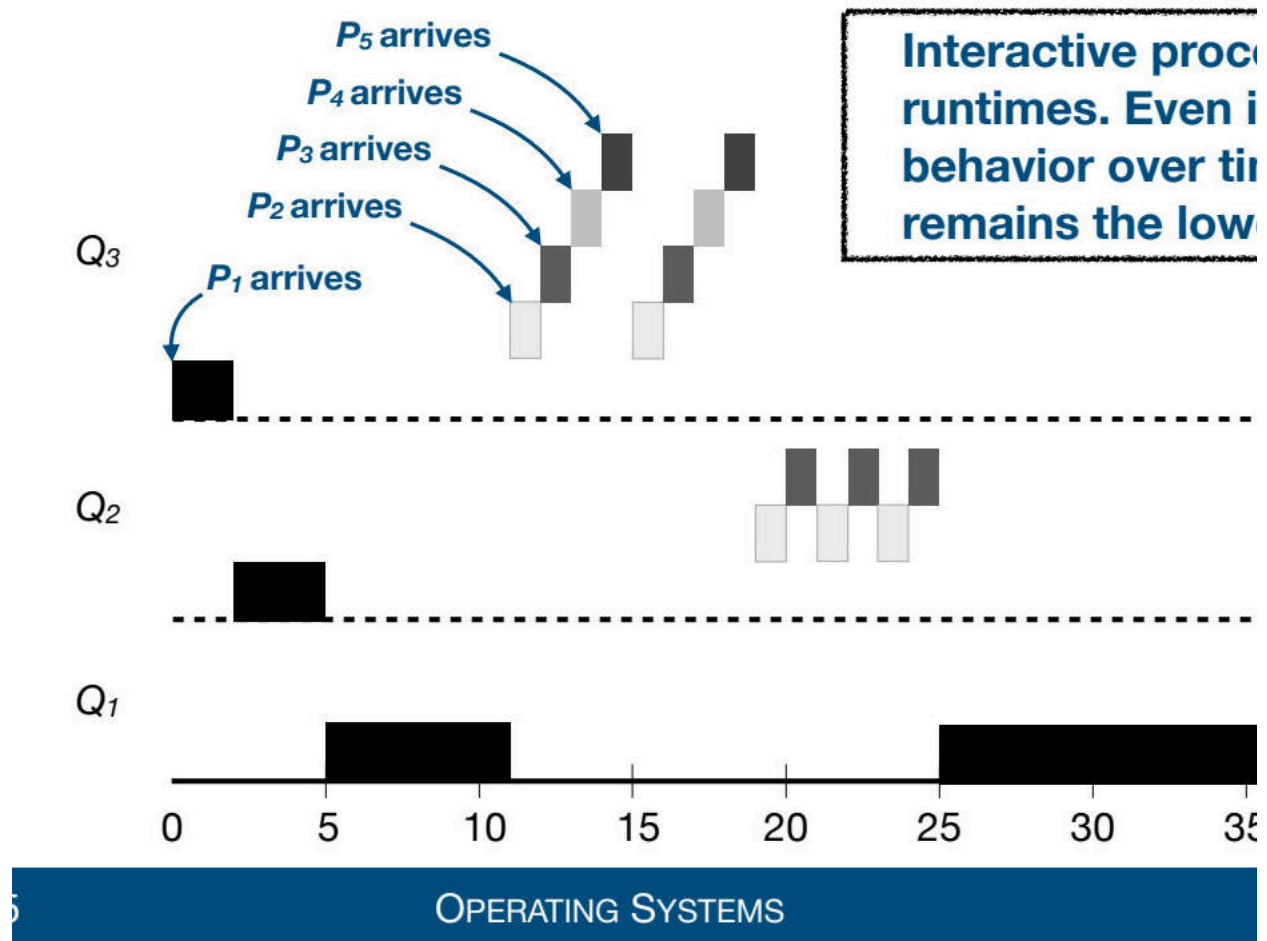
Regra 4 atualizada: Depois que um processo usa todo o seu tempo alocado em um determinado nível (independentemente de quantas vezes ele cedeu a CPU), sua prioridade é reduzida (ou seja, move-se para baixo uma fila).

Assim, o tempo alocado deve considerar a soma de todos os CPU bursts de um processo.



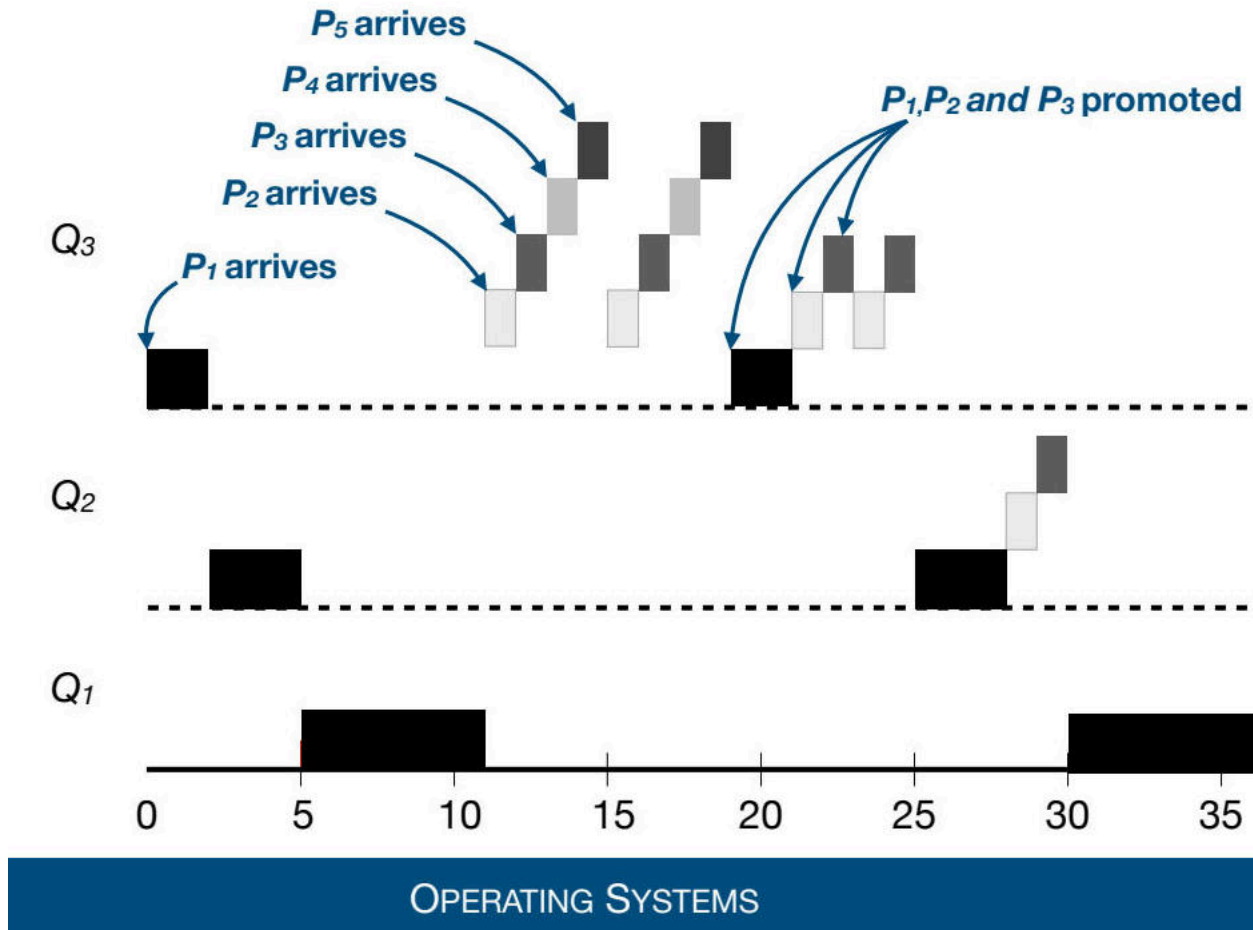
Inanição no MLFQ

A inanição de processos com tempos de execução mais longos ainda pode existir.



Reforço de Prioridade no MLFQ (Mecanismo de Envelhecimento)

Regra 5: Após um período de tempo S , mova todos os trabalhos para a fila superior.



Resumo do MLFQ

- Processos de curta duração são executados com alta prioridade (aproxima STCF).
 - Bom para o tempo de retorno.
- Processos de alta prioridade são frequentemente trocados (aproxima RR).
 - Bom para o tempo de resposta.

Desafio: vários botões de ajuste no MLFQ:

- Quantas filas?
- Quanto tempo deve ser o tempo alocado por fila?
- Quando o aumento de prioridade deve ser chamado?

Não há resposta fácil: é preciso conhecer o SO e as cargas de trabalho em execução nele.

Mais Informações

- Capítulos 7 e 8 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.
- Avi Silberschatz, Peter Baer Galvin, Greg Gagne. Operating System Concepts (10. ed). John Wiley & Sons, 2018.

Tópicos Interessantes (Não Abordados em Detalhe Neste Curso)

Proportional-Sharing

- Escalonadores de compartilhamento proporcional (também conhecidos como compartilhamento justo).
 - Objetivo: dar a cada trabalho uma porcentagem justa da CPU (métrica de justiça).
 - Algoritmos interessantes:
 - Escalonamento de Loteria
 - Escalonamento de Passos
 - Completely Fair Scheduler (CFS)
- CFS é o escalonador justo mais amplamente usado.
 - Usado por padrão em várias distribuições Linux.
 - Objetivos:
 - Ser justo entre os vários processos prontos para executar.
 - Ser muito eficiente ao escolher o próximo processo a ser executado.

CFS: Uma Breve Visão Geral

- **Tempo de execução virtual (vruntime):** Conta o tempo de CPU usado por um determinado processo.
 - Incrementado cada vez que um processo usa a CPU.
- **Critério de seleção:** Escolha o processo com o menor vruntime.
- O vruntime pode ser incrementado de forma diferente para processos específicos.
 - Exemplo: o tempo aumenta mais lentamente para processos de alta prioridade.
 - A prioridade pode ser escolhida por usuários e administradores de sistema (nível "nice" de um processo).
- Uma estrutura de árvore vermelho-preta é usada para indexar processos por seu vruntime e para escolher eficientemente o próximo a ser executado.

CFS: Uma Breve Visão Geral

- **Alvo de latência de escalonamento (sched_latency):** Dentro do intervalo de tempo sched_latency, cada processo deve ter uma chance justa de usar a CPU.
 - Exemplo: Supondo 4 processos e sched_latency = 48 ms, cada processo recebe uma fatia de tempo de 12 ms.
- A fatia de tempo é ajustada dinamicamente quando os processos chegam/saem do sistema.
 - Exemplo: Se 2 processos terminarem de executar, a fatia de tempo para cada um dos dois processos restantes é aumentada para 24 ms.
- Uma fatia de tempo mínima deve ser definida quando um grande número de processos está esperando para ser escalonado (min_granularity).

Tópicos Avançados (Não Abordados Neste Curso)

- Escalonamento de multiprocessadores
 - Sincronização
 - Afinidade de Cache